

Automated License Plate Detection and Anonymisation

A YOLOv8-based privacy pipeline for GDPR/CCPA-compliant image data

Objective	Detect every license plate in an image and irreversibly blur it, leaving the rest of the image usable
Data	Client dataset in YOLO format: 25,470 training, 1,073 validation and 386 test images
Model	YOLOv8s (single-stage detector), fine-tuned at 640 px
Anonymisation	Gaussian blur applied only inside each detected (and slightly enlarged) plate box

Executive Summary

- **Outcome.** The pipeline detects license plates in real-world street images and blurs them with a Gaussian kernel confined to each plate. On the held-out test set (386 images, 512 plates) the client model **fully covers 90.2% of all plates** at the deployment threshold, against 76.8% for a pilot model trained without client data, and **cuts plates left visible from 97 to 34**.
- **Detection quality.** Precision 0.920, recall 0.879, mAP@50 0.921 and mAP@50-95 0.682 (pilot: 0.794, 0.707, 0.744 and 0.442), at **2.9 ms per image** on an Apple M5 laptop GPU.
- **Model.** YOLOv8s fine-tuned on the client data for 90 epochs; the best checkpoint (epoch 47) is used. Validation scores levelled off from about epoch 40, so further gains will come from better training data rather than longer training.
- **Data.** The audit found and quarantined 155 duplicate images and 113 training images that leaked validation photos. Only about 19% of the training data resembles the real-world test images, which is the main lever for further improvement.
- **Recommendation.** Deploy with a low confidence threshold (0.1 to 0.2) so that uncertain plates are still blurred, route low-confidence detections to human review, and enrich training with real-world street photos.

1. Business Problem and Context

Organisations that collect street-level images and video (fleet cameras, insurers, mapping and autonomous-driving teams) hold large volumes of **personally identifiable information**. A license plate links a vehicle to its registered owner, so under GDPR and

CCPA it must be anonymised before the data is stored in the cloud, shared with partners or used to train AI models.

Manual redaction does not scale to tens of thousands of images, so the client needs an automated step that:

- 1. **Finds every plate**, including small, distant, angled and multiple plates in one image.
- 2. **Makes each plate unreadable**, to people and to OCR software, without damaging the rest of the image.
- 3. **Runs fast enough** to sit in a high-volume ingestion pipeline.

How success is measured

Metric	Why it matters
Recall / plates left visible	A missed plate is a privacy breach. This is the most important number.
Precision	A false detection only blurs a harmless region, so this matters less than recall.
mAP@50 and mAP@50-95	Standard detection quality; mAP@50-95 also rewards tight boxes.
Inference latency	Determines throughput and hardware cost.
Blur effectiveness	Blurred plates must not be readable by an OCR engine.

Approach

- 1. **Data audit and cleaning**: verify counts, label format, duplicates and train/validation leakage.
- 2. **Exploratory analysis**: data sources, plates per image and plate sizes, which drive the model settings.
- 3. **Coordinate verification**: convert YOLO labels to pixels and visually check them before training.
- 4. **Training**: fine-tune YOLOv8s on the client data at 640 px.
- 5. **Evaluation**: compare against a pilot model, and measure how many plates remain visible after blurring.
- 6. **Anonymisation**: Gaussian blur inside each detected box, verified with OCR.

2. Setup and Configuration

All paths and switches live in one cell. `RUN_TRAINING = False` reuses the trained weights so the notebook runs in minutes; set it to `True` to retrain from scratch (about 14 minutes per epoch on an Apple M5 GPU).

```
In [1]: %pip install -q ultralytics opencv-python pytesseract
```

Note: you may need to restart the kernel to use updated packages.

```
In [2]: import os
import re
import random
import shutil
import hashlib
from collections import Counter

import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

import torch
import ultralytics
from ultralytics import YOLO

# --- Configuration -----
DATA_ROOT = '.../License_Plate_Case_Study' # images/{split} and labels/{split}
SPLITS = ['train', 'val', 'test']
EXPECTED_IMAGES = {'train': 25470, 'val': 1073, 'test': 386} # counts stat

DATA_YAML = 'client_data.yaml'
RUN_PROJECT = os.path.abspath('runs/client')
TRAIN_RUN_NAME = 'yolov8s_90ep' # checkpoint evaluated in this report
PILOT_WEIGHTS = 'runs/detect/train-7/weights/best.pt' # pilot model trained
RUN_TRAINING = False

IMGSZ = 640
BLUR_CONF = 0.2 # low on purpose: a missed plate is a privacy breach, an
BLUR_PAD = 0.15 # each box is enlarged by 15% per side before blurring
SEED = 0

DEVICE = 'mps' if torch.backends.mps.is_available() else ('cuda' if torch.cuda
random.seed(SEED)
plt.rcParams['figure.dpi'] = 90
pd.set_option('display.width', 140)

print(f'Ultralytics {ultralytics.__version__} | torch {torch.__version__} |
```

Ultralytics 8.4.171 | torch 2.12.0 | device: mps

```
In [3]: def image_dir(split):
        return os.path.join(DATA_ROOT, 'images', split)

def label_dir(split):
    return os.path.join(DATA_ROOT, 'labels', split)

def stems(folder, ext):
    """Returns the file names (without extension) in `folder` that end with
```

```

    return {os.path.splitext(f)[0] for f in os.listdir(folder) if f.endswith(
def read_labels(label_path):
    """Reads a YOLO label file into a list of [class_id, x_center, y_center,
    if not os.path.exists(label_path):
        return []
    with open(label_path) as f:
        return [[float(v) for v in line.split()] for line in f if line.strip

def base_id(stem):
    """Strips the augmentation suffix added on export, e.g. 'abc_jpg.rf.123'
    return re.split(r'_jpg\.rf\. ', stem)[0]

def load_rgb(path):
    """Loads an image as RGB (OpenCV reads BGR, matplotlib expects RGB)."""
    return cv2.cvtColor(cv2.imread(path), cv2.COLOR_BGR2RGB)

```

3. Data Audit and Cleaning

Before any modelling we check that the delivered data matches the specification. The audit found three issues, which the next cell fixes **without deleting anything**: files are moved to quarantine folders next to the dataset so every change can be reviewed or reversed.

1. **Duplicate downloads**: files named `...(1).jpg` , `...(2).txt` are byte-identical copies created while downloading the archive in parts.
2. **Train/validation leakage**: some training images are augmented copies of photos that also appear in the validation set. The model would be scored on photos it has effectively seen, which inflates validation results.
3. **Label consistency**: every image must have exactly one label file with valid YOLO rows.

The cleaning cell is idempotent: once the dataset is clean, re-running it moves nothing.

```

In [4]: DUPLICATE_SUFFIX = re.compile(r'\\(\\d+\\)$') # 'abc(1)', 'abc(2)' ...

def file_md5(path):
    with open(path, 'rb') as f:
        return hashlib.md5(f.read()).hexdigest()

def quarantine(files, kind, split, reason):
    """Moves files from DATA_ROOT/<kind>/<split> to DATA_ROOT/_<reason>/<kind>
    dst = os.path.join(DATA_ROOT, f'_{reason}', kind, split)
    os.makedirs(dst, exist_ok=True)
    for f in files:
        shutil.move(os.path.join(DATA_ROOT, kind, split, f), os.path.join(ds

```

```

    return len(files)

moved = Counter()
for kind, folder in [('images', image_dir('train')), ('labels', label_dir('train'))]:
    # 1. Duplicate downloads: quarantine only copies that are byte-identical
    duplicates = []
    for f in os.listdir(folder):
        stem, ext = os.path.splitext(f)
        if DUPLICATE_SUFFIX.search(stem):
            original = os.path.join(folder, DUPLICATE_SUFFIX.sub('', stem) + ext)
            if os.path.exists(original) and file_md5(original) == file_md5(f):
                duplicates.append(f)
    moved[('Duplicate downloads', kind)] += quarantine(duplicates, kind, 'train')

# 2. Leakage: training copies of photos that are also in the validation set
val_bases = {base_id(s) for s in stems(image_dir('val'), '.jpg')}
for kind, folder, ext in [('images', image_dir('train'), '.jpg'), ('labels', label_dir('train'), '.jpg')]:
    leaking = [f for f in os.listdir(folder) if f.endswith(ext) and base_id(f) in val_bases]
    moved[('Train copies of validation photos', kind)] += quarantine(leaking, kind, 'train')

def quarantined(reason, kind):
    path = os.path.join(DATA_ROOT, f'_{reason}', kind, 'train')
    return len(os.listdir(path)) if os.path.isdir(path) else 0

cleaning_log = pd.DataFrame([
    {'issue': issue, 'files': kind, 'moved in this run': moved[(issue, kind)],
     'total in quarantine': quarantined(reason, kind)}
    for issue, reason in [('Duplicate downloads', 'duplicates_removed'),
                          ('Train copies of validation photos', 'leakage_removed')]
    for kind in ['images', 'labels']
])
cleaning_log

```

Out[4]:

	issue	files	moved in this run	total in quarantine
0	Duplicate downloads	images	0	155
1	Duplicate downloads	labels	0	202
2	Train copies of validation photos	images	0	113
3	Train copies of validation photos	labels	0	113

In [5]:

```

final_train = len(stems(image_dir('train'), '.jpg'))
dup = quarantined('duplicates_removed', 'images')
leak = quarantined('leakage_removed', 'images')
delivered = final_train + dup + leak

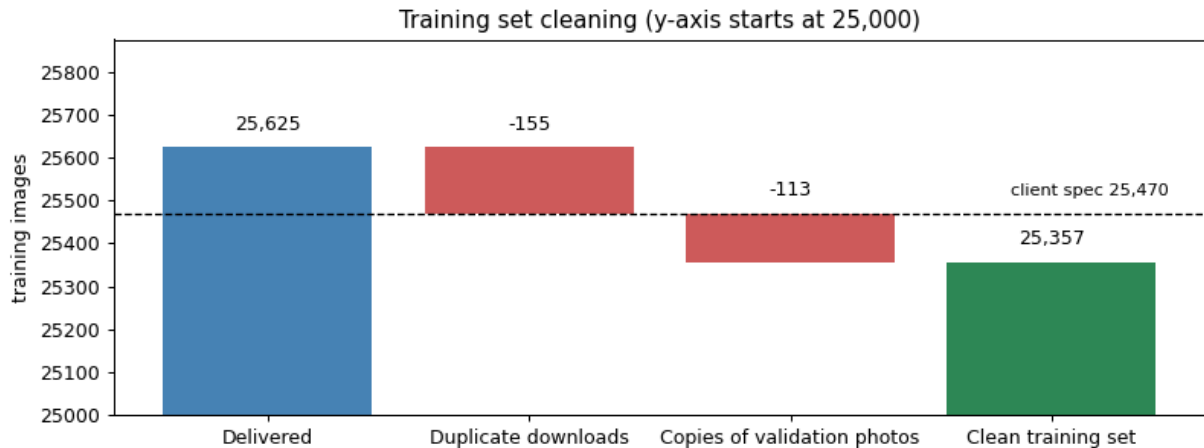
steps = ['Delivered', 'Duplicate downloads', 'Copies of validation photos',
         'Train copies of validation photos']
values = [delivered, -dup, -leak, final_train]
bottoms = [0, delivered - dup, delivered - dup - leak, 0]
heights = [delivered, dup, leak, final_train]
colors = ['steelblue', 'indianred', 'indianred', 'seagreen']

```

```

fig, ax = plt.subplots(figsize=(9, 3.5))
ax.bar(steps, heights, bottom=bottoms, color=colors)
for i, v in enumerate(values):
    ax.text(i, bottoms[i] + heights[i] + 40, f'{v:+,}' if 0 < i < 3 else f'{v:,}')
ax.axhline(EXPECTED_IMAGES['train'], color='black', linestyle='--', linewidth=1)
ax.text(3.45, EXPECTED_IMAGES['train'] + 40, f'client spec {EXPECTED_IMAGES["train"]}')
ax.set_ylim(25000, delivered + 250)
ax.set_ylabel('training images')
ax.set_title('Training set cleaning (y-axis starts at 25,000)')
plt.tight_layout()
plt.show()

```



```

In [6]: audit_rows = []
for split in SPLITS:
    imgs, lbls = stems(image_dir(split), '.jpg'), stems(label_dir(split), '.txt')
    labels = [read_labels(os.path.join(label_dir(split), s + '.txt')) for s in labels]
    boxes = [b for rows in labels for b in rows]
    audit_rows.append({
        'split': split,
        'images (client spec)': EXPECTED_IMAGES[split],
        'images (on disk)': len(imgs),
        'label files': len(lbls),
        'images without label': len(imgs - lbls),
        'labels without image': len(lbls - imgs),
        'plates': len(boxes),
        'values per row': sorted({len(b) for b in boxes}),
        'class ids': sorted({int(b[0]) for b in boxes}),
        'coords outside [0,1]': sum(any(v < 0 or v > 1 for v in b[1:]) for b in boxes),
        'empty label files': sum(len(rows) == 0 for rows in labels),
    })
audit = pd.DataFrame(audit_rows).set_index('split')
audit.T

```

Out [6]:

	split	train	val	test
images (client spec)	25470	1073	386	
images (on disk)	25357	1073	386	
label files	25357	1073	386	
images without label	0	0	0	
labels without image	0	0	0	
plates	28067	1573	512	
values per row	[5]	[5]	[5]	
class ids	[0]	[0]	[0]	
coords outside [0,1]	0	0	0	
empty label files	17	0	0	

In [7]: *# Cross-split leakage after cleaning: no original photo may appear in two splits*

```

split_bases = {split: {base_id(s) for s in stems(image_dir(split), '.jpg')}}
pd.DataFrame([
    {'pair': f'{a} / {b}', 'shared photos': len(split_bases[a] & split_bases[b])
     for a, b in [('train', 'val'), ('train', 'test'), ('val', 'test')]}
]).set_index('pair')

```

Out [7]:

	shared photos
pair	
train / val	0
train / test	0
val / test	0

Audit outcome. After cleaning, the training set holds 25,357 images (the 25,470 stated by the client minus 113 copies of validation photos), and every split has one valid label file per image. Labels use the standard five-value YOLO row (`class` `x_center` `y_center` `width` `height`) with a single class (`0 = license_plate`), and no split shares a photo with another. The 17 empty training labels are valid background images.

4. Exploratory Data Analysis

The analysis below answers three practical questions: where the data comes from, how many plates an image contains, and how large the plates are. Each answer feeds a modelling decision in Section 7.

In [8]:

```

SOURCE_RULES = [
    (r'^[0-9a-f]{16}$', 'Real-world street photos'),
    (r'^CarLongPlateGen', 'Long car plates (noise-augmented)'),

```

```

(r'^CarLongPlate', 'Long car plates'),
(r'^xemayBigPlate', 'Motorbike plates (large)'),
(r'^xemay', 'Motorbike plates'),
(r'quandoi', 'Military plates'),
(r'^Cars\d', 'Car close-ups'),
]

def source_of(stem):
    """Groups an image into a data source using its file-name pattern."""
    name = base_id(stem)
    for pattern, source in SOURCE_RULES:
        if re.search(pattern, name):
            return source
    return 'Other'

image_rows, plate_rows = [], []
for split in SPLITS:
    for stem in sorted(stems(image_dir(split), '.jpg')):
        with Image.open(os.path.join(image_dir(split), stem + '.jpg')) as im:
            img_w, img_h = im.size # reads the header only, not the pixels
            labels = read_labels(os.path.join(label_dir(split), stem + '.txt'))
            image_rows.append({'split': split, 'stem': stem, 'source': source_of(
                'augmented': '_jpg.rf.' in stem, 'img_w': img_w,
                'n_plates': len(labels)})
            for _, xc, yc, w, h in labels:
                plate_rows.append({'split': split, 'stem': stem, 'img_w': img_w,
                    'x_center': xc, 'y_center': yc, 'width': w,
                    'width_px': w * img_w, 'height_px': h * img_h})

images_df = pd.DataFrame(image_rows)
plates_df = pd.DataFrame(plate_rows)
print(f'{len(images_df):,} images and {len(plates_df):,} plates loaded')

```

26,816 images and 30,152 plates loaded

4.1 Data sources and augmentation

```

In [9]: source_mix = pd.crosstab(images_df['source'], images_df['split'], normalize=
source_mix = source_mix[SPLITS].sort_values('train', ascending=False)

train = images_df[images_df['split'] == 'train']
augmentation = pd.Series({
    'training images': len(train),
    'unique source photos': train['stem'].map(base_id).nunique(),
    'augmented copies': int(train['augmented'].sum()),
})

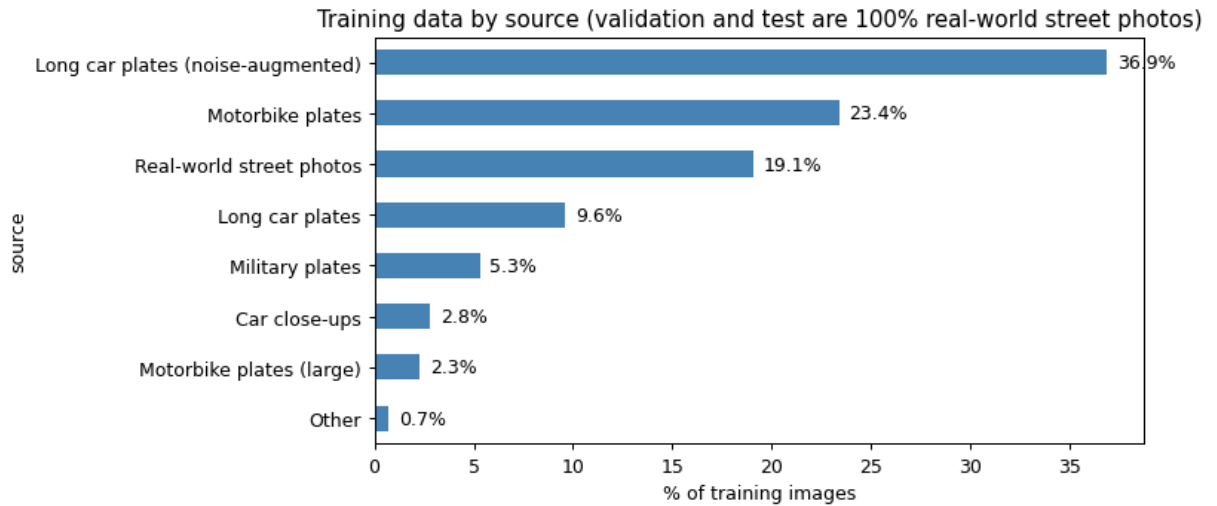
fig, ax = plt.subplots(figsize=(9, 4))
source_mix['train'].sort_values().plot.barh(ax=ax, color='steelblue')
ax.set_xlabel('% of training images')
ax.set_title('Training data by source (validation and test are 100% real-world)')
for i, v in enumerate(source_mix['train'].sort_values()):
    ax.text(v + 0.5, i, f'{v:.1f}%', va='center')

```



```
plt.tight_layout()
plt.show()

display(source_mix)
display(augmentation.to_frame('count'))
```



	split	train	val	test
source				
Long car plates (noise-augmented)		36.9	0.0	0.0
Motorbike plates		23.4	0.0	0.0
Real-world street photos		19.1	100.0	100.0
Long car plates		9.6	0.0	0.0
Military plates		5.3	0.0	0.0
Car close-ups		2.8	0.0	0.0
Motorbike plates (large)		2.3	0.0	0.0
Other		0.7	0.0	0.0

	count
training images	25357
unique source photos	12847
augmented copies	21062

Findings

- The training set combines several sources. Only about one fifth of it are real-world street photos, the same kind of image as the validation and test sets. The rest are close-up car plates (many with added noise), motorbike plates and military plates, mostly from Vietnam.

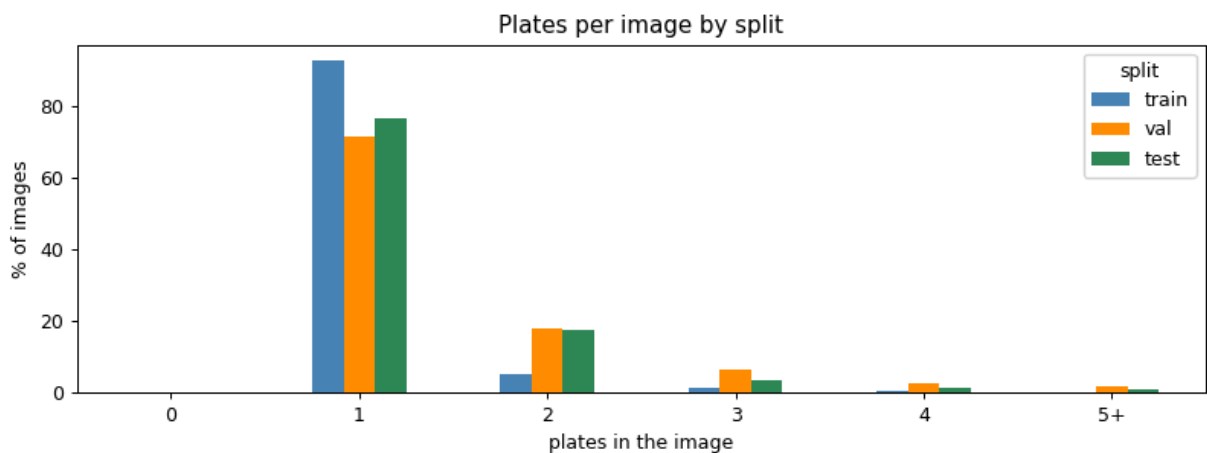
- About 83% of training files are augmented copies (rotations, brightness and border changes) of roughly 12,850 unique photos. The effective variety is therefore about half the headline count.
- **Implication:** the model is evaluated on a different mix than it is trained on (*domain shift*). Test results are a realistic, slightly conservative estimate of field performance, and adding more real-world street photos is the most valuable next data investment.

4.2 Plates per image

```
In [10]: bins = pd.cut(images_df['n_plates'], [-1, 0, 1, 2, 3, 4, 1000], labels=['0',
plates_per_image = pd.crosstab(bins, images_df['split'], normalize='columns')

ax = plates_per_image.plot.bar(figsize=(9, 3.5), rot=0, color=['steelblue',
ax.set_xlabel('plates in the image')
ax.set_ylabel('% of images')
ax.set_title('Plates per image by split')
plt.tight_layout()
plt.show()

multi = (images_df['n_plates'] > 1).groupby(images_df['split']).mean().mul(100)
print('Images with more than one plate (%):', multi.to_dict())
```



Images with more than one plate (%): {'train': 7.2, 'val': 28.6, 'test': 23.3}

Findings. About 23% of test images and 29% of validation images contain **more than one plate** (up to 19 in one image), compared with 7% in training. The pipeline must therefore blur every detection rather than only the most confident one, and a model that finds only the obvious plate per image would leak a large share of plates.

4.3 Image resolution and plate size

YOLO resizes every image so that its longer side equals the training size (`imgsz`). A plate's *effective* width is therefore its pixel width scaled by `imgsz / longer side`. Plates that shrink below roughly 16 px are hard for any detector.

```

In [11]: longer_side = plates_df[['img_w', 'img_h']].max(axis=1)
for size in [320, 640]:
    plates_df[f'width_at_{size}'] = plates_df['width_px'] * size / longer_si

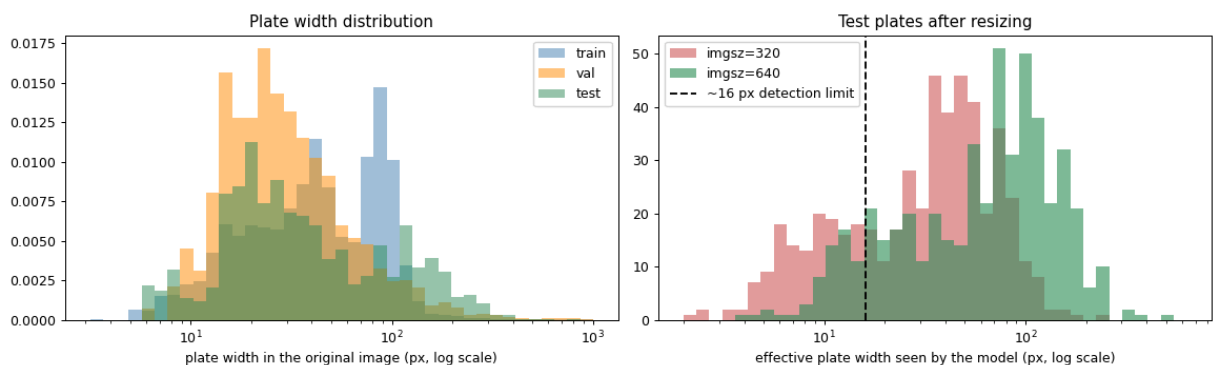
resolution = images_df.groupby('split')[['img_w', 'img_h']].median().loc[SPLITS]
size_table = plates_df.groupby('split').agg(
    median_width_px=('width_px', 'median'),
    p10_width_px=('width_px', lambda s: s.quantile(0.10)),
    pct_below_16px_at_320=('width_at_320', lambda s: (s < 16).mean() * 100),
    pct_below_16px_at_640=('width_at_640', lambda s: (s < 16).mean() * 100),
).loc[SPLITS].round(1)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for split, color in zip(SPLITS, ['steelblue', 'darkorange', 'seagreen']):
    widths = plates_df.loc[plates_df['split'] == split, 'width_px']
    axes[0].hist(widths, bins=np.logspace(0.5, 3, 40), alpha=0.5, density=True)
axes[0].set_xscale('log')
axes[0].set_xlabel('plate width in the original image (px, log scale)')
axes[0].set_title('Plate width distribution')
axes[0].legend()

test_plates = plates_df[plates_df['split'] == 'test']
for size, color in [(320, 'indianred'), (640, 'seagreen')]:
    axes[1].hist(test_plates[f'width_at_{size}'], bins=np.logspace(0.3, 2.8, 20),
                  label=f'imgsz={size}', color=color)
axes[1].axvline(16, color='black', linestyle='--', label='~16 px detection limit')
axes[1].set_xscale('log')
axes[1].set_xlabel('effective plate width seen by the model (px, log scale)')
axes[1].set_title('Test plates after resizing')
axes[1].legend()
plt.tight_layout()
plt.show()

display(resolution.rename(columns={'img_w': 'median image width', 'img_h': 'median image height'}))
display(size_table)

```



	median image width	median image height
split		
train	416.0	416.0
val	1024.0	768.0
test	1024.0	768.0

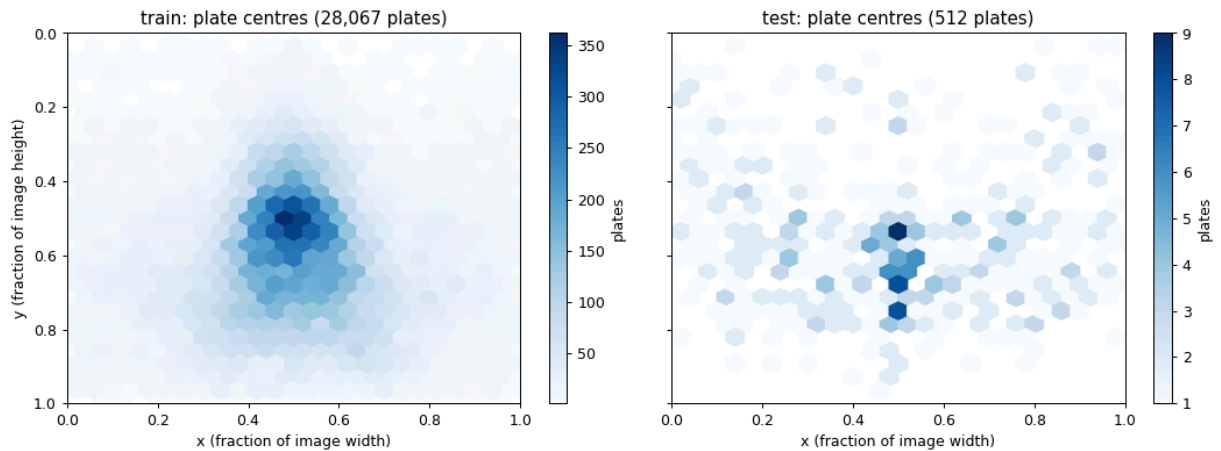
	median_width_px	p10_width_px	pct_below_16px_at_320	pct_below_16px_at_640
split				
train	79.5	27.5	13.9	4.7
val	52.5	19.8	49.4	19.4
test	113.7	22.7	27.9	11.7

Findings

- Training images are mostly 416 × 416 exports, while validation and test images are full photos of about 1024 × 768 in which plates are often small: one in ten test plates is narrower than about 23 px.
- At `imgsz=320`, 28% of test plates (and 49% of validation plates) would shrink below 16 px. At `imgsz=640` this falls to 12% (19% for validation).
- **Decision:** train and run inference at **640 px**. Halving the resolution would make roughly one plate in four effectively invisible to the model, which is unacceptable for a privacy control.

4.4 Where plates appear in the image

```
In [12]: fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), sharey=True)
for ax, split in zip(axes, ['train', 'test']):
    p = plates_df[plates_df['split'] == split]
    hb = ax.hexbin(p['x_center'], p['y_center'], gridsize=25, extent=(0, 1,
    ax.set_xlim(0, 1)
    ax.set_ylim(1, 0) # image coordinates: y grows downwards
    ax.set_title(f'{split}: plate centres ({len(p):,} plates)')
    ax.set_xlabel('x (fraction of image width)')
    fig.colorbar(hb, ax=ax, label='plates')
axes[0].set_ylabel('y (fraction of image height)')
plt.tight_layout()
plt.show()
```



Finding. In training, plates cluster near the centre of the frame (close-up exports); in the test set they appear anywhere in the scene, including the edges. Mosaic augmentation (Section 7.4) moves plates to random positions during training, which helps close this gap.

5. Coordinate Transformation: YOLO to Pixel Coordinates

YOLO stores each box as **normalised centre and size**: `x_center`, `y_center`, `width`, `height`, each between 0 and 1 relative to the image. OpenCV draws rectangles from **pixel corners** `(x1, y1)` (top-left) and `(x2, y2)` (bottom-right). The conversion is:

$$\begin{aligned} x_1 &= (x_c - w/2) \cdot W & y_1 &= (y_c - h/2) \cdot H \\ x_2 &= (x_c + w/2) \cdot W & y_2 &= (y_c + h/2) \cdot H \end{aligned}$$

where W and H are the image width and height in pixels. Results are rounded to whole pixels and clipped to the image, because a box touching the border can otherwise round to a coordinate just outside it.

```
In [13]: def yolo_to_pixel(x_center, y_center, width, height, img_w, img_h):
        """
        Converts a normalised YOLO box to integer pixel corners (x1, y1, x2, y2)
        clipped to the image.
        """
        x1 = (x_center - width / 2) * img_w
        y1 = (y_center - height / 2) * img_h
        x2 = (x_center + width / 2) * img_w
        y2 = (y_center + height / 2) * img_h
        x1, x2 = np.clip([x1, x2], 0, img_w - 1)
        y1, y2 = np.clip([y1, y2], 0, img_h - 1)
        return int(round(x1)), int(round(y1)), int(round(x2)), int(round(y2))

        def pixel_to_yolo(x1, y1, x2, y2, img_w, img_h):
```

```
"""Inverse of yolo_to_pixel: pixel corners back to a normalised YOLO box
return ((x1 + x2) / 2 / img_w, (y1 + y2) / 2 / img_h, (x2 - x1) / img_w,
```

```
In [14]: # Worked example on the first test label
row = plates_df[plates_df['split'] == 'test'].iloc[0]
W, H = int(row['img_w']), int(row['img_h'])
xc, yc, w, h = row['x_center'], row['y_center'], row['width'], row['height']
x1, y1, x2, y2 = yolo_to_pixel(xc, yc, w, h, W, H)

pd.DataFrame([
    ('Image size (W x H)', f'{W} x {H}'),
    ('YOLO label (xc, yc, w, h)', f'{xc:.4f}, {yc:.4f}, {w:.4f}, {h:.4f}'),
    ('x1 = (xc - w/2) * W', f'({xc:.4f} - {w / 2:.4f}) * {W} = {x1}'),
    ('y1 = (yc - h/2) * H', f'({yc:.4f} - {h / 2:.4f}) * {H} = {y1}'),
    ('x2 = (xc + w/2) * W', f'({xc:.4f} + {w / 2:.4f}) * {W} = {x2}'),
    ('y2 = (yc + h/2) * H', f'({yc:.4f} + {h / 2:.4f}) * {H} = {y2}'),
], columns=['step', 'value']).set_index('step')
```

Out[14]:

	value
step	
Image size (W x H)	1024 x 683
YOLO label (xc, yc, w, h)	0.2690, 0.4399, 0.1218, 0.1441
x1 = (xc - w/2) * W	(0.2690 - 0.0609) * 1024 = 213
y1 = (yc - h/2) * H	(0.4399 - 0.0721) * 683 = 251
x2 = (xc + w/2) * W	(0.2690 + 0.0609) * 1024 = 338
y2 = (yc + h/2) * H	(0.4399 + 0.0721) * 683 = 350

```
In [15]: # Verification: convert every validation and test plate to pixels and back.
# Any error larger than rounding (about 1 px) would mean the conversion is w
errors = []
for _, r in plates_df[plates_df['split'].isin(['val', 'test'])].iterrows():
    W, H = r['img_w'], r['img_h']
    corners = yolo_to_pixel(r['x_center'], r['y_center'], r['width'], r['hei
    xc, yc, w, h = pixel_to_yolo(*corners, W, H)
    errors.append(max(abs(xc - r['x_center']) * W, abs(w - r['width']) * W,
                      abs(yc - r['y_center']) * H, abs(h - r['height']) * H)

errors = np.array(errors)
print(f'Plates checked: {len(errors)} | max round-trip error: {errors.max():
      f'within 1 px: {(errors <= 1).mean():.1%}')
```

Plates checked: 2085 | max round-trip error: 1.31 px | within 1 px: 99.9%

Every box survives the round trip within rounding error (boxes that touch the image border can differ by slightly more because of clipping, which is the intended behaviour).

6. Visual Label Check Before Training

```
In [16]: def draw_yolo_boxes(image_rgb, labels, color=(0, 255, 0), thickness=2):
    """Returns a copy of the image with every YOLO label drawn as a rectangle
    out = image_rgb.copy()
    img_h, img_w = out.shape[:2]
    for _, xc, yc, w, h in labels:
        x1, y1, x2, y2 = yolo_to_pixel(xc, yc, w, h, img_w, img_h)
        cv2.rectangle(out, (x1, y1), (x2, y2), color, thickness)
    return out

def pick(split, source=None, min_plates=1, n=1):
    """Picks n random images from a split, optionally filtered by source and
    pool = images_df[(images_df['split'] == split) & (images_df['n_plates']
    if source:
        pool = pool[pool['source'] == source]
    return pool.sample(n, random_state=SEED)['stem'].tolist()

examples = ([('train', s) for s in pick('train', 'Real-world street photos')]
            + [('train', s) for s in pick('train', 'Motorbike plates')]
            + [('train', s) for s in pick('train', 'Long car plates (noise-augmented)')]
            + [('test', s) for s in pick('test', min_plates=3, n=3)])

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, (split, stem) in zip(axes.flat, examples):
    image = load_rgb(os.path.join(image_dir(split), stem + '.jpg'))
    labels = read_labels(os.path.join(label_dir(split), stem + '.txt'))
    ax.imshow(draw_yolo_boxes(image, labels, thickness=max(2, image.shape[1]
    ax.set_title(f'{split} | {source_of(stem)} | {len(labels)} plate(s)', font
    ax.axis('off')
plt.tight_layout()
plt.show()
```



Why this step is critical before training. A detector learns exactly what its labels say, so label errors become model errors. Overlaying the labels takes seconds and catches

problems that metrics cannot:

- **Format errors:** swapped x/y, pixel values stored instead of normalised ones, or corner format instead of centre format all produce boxes that miss the plate, and training would still run without any error message.
- **Incomplete annotation:** an unlabeled plate teaches the model that plates are *background*. In our pilot study, the first label conversion kept only one plate per image and silently dropped 38 of 471 plates (8%). This was only spotted by inspecting multi-plate images.
- **Cost control:** one training run on this dataset takes about three hours. Verifying labels first is far cheaper than discovering a data problem after training.

The overlays above confirm that boxes sit tightly on the plates in every source, including small, distant and multiple plates.

7. Model Design Decisions

7.1 Why YOLOv8 instead of a two-stage detector?

Two-stage detectors (for example Faster R-CNN) first propose candidate regions and then classify each one. YOLO predicts boxes and classes for the whole image **in a single forward pass**.

Criterion	YOLOv8 (single-stage)	Two-stage (e.g. Faster R-CNN)
Speed	A few ms per image on a laptop GPU	Typically several times slower
Accuracy on one, well-defined class	High; a plate is a rigid, high-contrast rectangle	Slightly better on crowded, multi-class scenes
Deployment	Exports to ONNX, TensorRT and CoreML for edge devices	Heavier, harder to run at the edge
Tooling	Training, validation and export in one library	More engineering effort

The task is a single, visually distinctive object class at high volume, so the extra accuracy of a two-stage detector does not justify its cost. YOLOv8 also supports the client's goal of anonymising "at the source" on cameras and edge devices.

7.2 Which YOLOv8 variant?

YOLOv8 comes in five sizes (n, s, m, l, x). We selected **YOLOv8s** (about 11 million parameters):

- In the pilot study, the nano model was adequate on close-up, centred plates, but the client data is harder: many small, distant and multiple plates in full-resolution street scenes. The small model's extra capacity helps on exactly these cases.
- YOLOv8s still runs in a few milliseconds per image at 640 px (measured in Section 9), so it fits a high-volume pipeline without a dedicated server GPU.
- Medium and larger variants would multiply training time (already about 14 minutes per epoch on 25K images) for gains that matter less than data quality, which Section 4 identified as the main lever.

7.3 How plate characteristics shaped the model scale

Plate characteristic (from Section 4)	Decision
Small: one in ten test plates is narrower than about 23 px	640 px input ; 320 px would push 28% of test plates below the detection limit
Wide and short, with fine text	Aspect-ratio-preserving resize (letterbox) so plates are not squashed
Often several per image (23% of test images)	Keep all detections; multi-plate images matter in evaluation
Rigid, high-contrast, single class	A small model (YOLOv8s) is sufficient; no need for m/l/x
Varied position, scale and lighting	Mosaic and colour augmentation during training

7.4 Preprocessing without losing fine plate detail

- **Letterbox resizing**: the longer side is scaled to 640 px and the shorter side is padded, so the plate's shape and text are never stretched or cropped.
- **Resolution chosen from data**: 640 px was selected from the plate-size analysis rather than by default.
- **No lossy steps**: no grayscale conversion, sharpening or compression; the model sees the original colour image.
- **Mosaic augmentation** combines four images into one, which places plates at smaller scales and random positions, exactly the hard cases in the test set. It is switched off for the final ten epochs (`close_mosaic=10`) so the model finishes on realistic, unaltered images.
- **Clean inputs**: duplicates and leaking copies were removed (Section 3) so the model is neither over-weighted on repeated images nor evaluated on photos it has seen.

8. Training

```
In [17]: # Dataset definition for Ultralytics; it finds each label by replacing /imag
dataset_yaml = f"""
path: {DATA_ROOT}
train: images/train
```

```

val: images/val
test: images/test

nc: 1
names: ['license_plate']
"""
with open(DATA_YAML, 'w') as f:
    f.write(dataset_yaml)
print(dataset_yaml)

```

```

path: ../License_Plate_Case_Study
train: images/train
val: images/val
test: images/test
nc: 1
names: ['license_plate']

```

In [18]: `TRAINED_WEIGHTS = os.path.join(RUN_PROJECT, TRAIN_RUN_NAME, 'weights', 'best`

```

if RUN_TRAINING:
    model = YOLO('yolov8s.pt') # COCO-pretrained starting point
    model.train(
        data=DATA_YAML,
        epochs=90,
        patience=0, # 0 disables early stopping so the full schedule runs
        close_mosaic=10, # mosaic augmentation on for the first 80 epochs,
        batch=16,
        imgsz=IMGSZ,
        device=DEVICE,
        cache=False, # 25K images at 640 px are too many to hold in memory
        seed=SEED,
        project=RUN_PROJECT,
        name=TRAIN_RUN_NAME,
        exist_ok=True,
    )

model = YOLO(TRAINED_WEIGHTS)
print('Using weights:', TRAINED_WEIGHTS)

```

Using weights: ../weights/best.pt

In [19]: `history = pd.read_csv(os.path.join(RUN_PROJECT, TRAIN_RUN_NAME, 'results.csv')`
`history.columns = history.columns.str.strip()`
`epoch_minutes = history['time'].diff().fillna(history['time']).div(60)`

```

fig, axes = plt.subplots(1, 3, figsize=(16, 4))
axes[0].plot(history['epoch'], history['train/box_loss'], '-', label='train')
axes[0].plot(history['epoch'], history['val/box_loss'], '-', label='validation')
axes[0].set_title('Box loss (lower is better)')
axes[1].plot(history['epoch'], history['metrics/precision(B)'], '-', label='precision')
axes[1].plot(history['epoch'], history['metrics/recall(B)'], '-', label='recall')
axes[1].set_title('Validation precision and recall')
axes[2].plot(history['epoch'], history['metrics/mAP50(B)'], '-', label='mAP@50')
axes[2].plot(history['epoch'], history['metrics/mAP50-95(B)'], '-', label='mAP@50-95')
axes[2].set_title('Validation mAP')
for ax in axes:
    ax.set_xlabel('epoch')

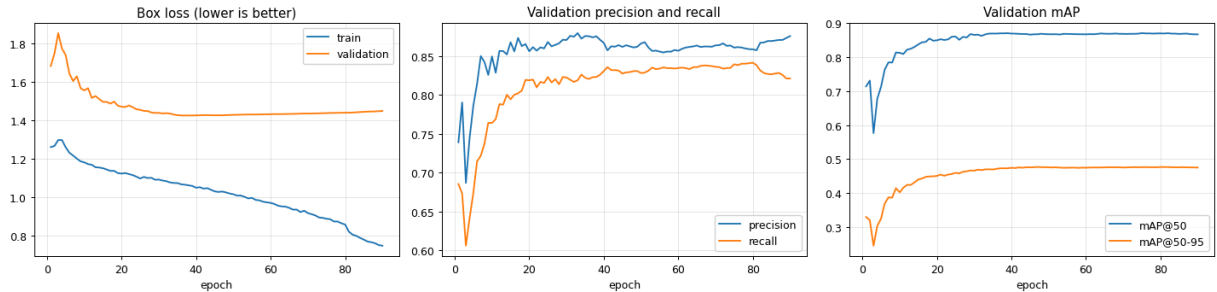
```

```

ax.grid(alpha=0.3)
ax.legend()
plt.tight_layout()
plt.show()

best = history.loc[history['metrics/mAP50-95(B)'].idxmax()]
print(f'Epochs: {len(history)} | typical (median) {epoch_minutes.median():.1f} min per epoch | best epoch {int(best["epoch"])} (mAP@50-95 {best["metrics/mAP50-95(B)"]})')

```



Epochs: 90 | typical (median) 14.2 min per epoch | best epoch 47 (mAP@50-95 0.477)

Training progress. The model was trained for 90 epochs, about 21 hours on an Apple M5 laptop GPU. Validation mAP@50 rose from 0.71 after the first epoch to 0.87, and mAP@50-95 from 0.33 to 0.477. The dip around epoch 3 is the warm-up phase, in which the learning rate ramps up.

- **Scores levelled off from about epoch 40.** Between epochs 40 and 90, validation mAP@50-95 changed by less than 0.005, while training loss kept falling and validation loss rose slightly, an early sign of overfitting. Training longer on the same data would therefore not improve the model.
- **Final ten epochs without mosaic.** Switching off mosaic augmentation raised precision slightly and lowered recall slightly, with no net gain in mAP.
- **Checkpoint used.** The report uses the best checkpoint (epoch 47), selected on validation mAP@50-95. A check on the test set confirmed that it also covers more plates than the final epoch-90 weights (90.2% against 89.1% fully covered), so the choice also holds for the business metric.

9. Evaluation on the Held-Out Test Set

All results in this section use the **386 test images (512 plates)**, which were never used for training or model selection. We compare two models:

- **Pilot model:** YOLOv8 trained earlier on a small public dataset of 433 images, without any client data. It shows what an off-the-shelf approach would achieve.
- **Client model:** YOLOv8s fine-tuned on the cleaned client training set (Section 8).

```
In [20]: pilot_model = YOLO(PILOT_WEIGHTS)
```

```
def evaluate(m, name):
    """Standard Ultralytics metrics on the test split."""
    r = m.val(data=DATA_YAML, split='test', imgsz=IMGSZ, device=DEVICE, batch_size=1,
              verbose=False, project=RUN_PROJECT, name=f'eval_{name}', exist_ok=True)
    return {'model': name, 'precision': r.box.mp, 'recall': r.box.mr, 'mAP@50': r.box.map50,
            'mAP@50-95': r.box.map, 'inference ms / image': r.speed['inference ms / image']}

comparison = pd.DataFrame([evaluate(pilot_model, 'Pilot model'),
                           evaluate(model, 'Client model')]).set_index('model')
comparison
```

Ultralytics 8.4.171 🚀 Python-3.14.5 torch-2.12.0 MPS (Apple M5 Pro)
Model summary (fused): 72 layers, 11,125,971 parameters, 0 gradients, 28.4 G FLOPs

	all	386	512	0.794	0.707	0.74
4	0.442					

Speed: 0.3ms preprocess, 3.9ms inference, 0.0ms loss, 5.4ms postprocess per image

Ultralytics 8.4.171 🚀 Python-3.14.5 torch-2.12.0 MPS (Apple M5 Pro)
Model summary (fused): 72 layers, 11,125,971 parameters, 0 gradients, 28.4 G FLOPs

	all	386	512	0.92	0.879	0.92
1	0.682					

Speed: 0.2ms preprocess, 2.9ms inference, 0.0ms loss, 3.1ms postprocess per image

Out[20]: precision recall mAP@50 mAP@50-95 inference ms / image

model					
Pilot model	0.794	0.707	0.744	0.442	3.880
Client model	0.920	0.879	0.921	0.682	2.941

```
In [21]: ax = comparison[['precision', 'recall', 'mAP@50', 'mAP@50-95']].T.plot.bar(
        figsize=(9, 4), rot=0, color=['lightgray', 'steelblue'])
ax.set_ylim(0, 1)
ax.set_title('Pilot model vs client model on the test set')
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f', fontsize=8)
plt.tight_layout()
plt.show()
```



9.1 Privacy view: how many plates stay visible?

Standard metrics average over all confidence thresholds. For anonymisation the relevant question is concrete: **at the threshold we deploy, how many real plates end up fully covered by a blur box?**

For every labelled test plate we measure what share of its area is covered by the enlarged blur boxes, and classify it as fully covered (90% or more), partly covered, or missed (under 10%).

```
In [22]: test_stems = sorted(images_df.loc[images_df['split'] == 'test', 'stem'])

def gt_boxes(stem):
    """Labelled plates of a test image as pixel corners."""
    row = images_df[(images_df['split'] == 'test') & (images_df['stem'] == stem)]
    labels = read_labels(os.path.join(label_dir('test'), stem + '.txt'))
    return [yolo_to_pixel(xc, yc, w, h, row['img_w'], row['img_h']) for _, x in labels.items()]

def predict_test(m, conf=0.01):
    """Runs the model once on every test image; thresholds are applied after prediction"""
    preds = {}
    for stem in test_stems:
        r = m.predict(os.path.join(image_dir('test'), stem + '.jpg'), conf=conf,
                      imgsz=IMGSZ, device=DEVICE, verbose=False)[0]
        preds[stem] = (r.boxes.xyxy.cpu().numpy(), r.boxes.conf.cpu().numpy())
    return preds

def pad_box(box, pad, img_w, img_h):
    """Enlarges a box by `pad` of its size on each side, clipped to the image"""
    x1, y1, x2, y2 = box
    dx, dy = (x2 - x1) * pad, (y2 - y1) * pad
    return (max(0, int(x1 - dx)), max(0, int(y1 - dy)), min(img_w, int(x2 + dx)), min(img_h, int(y2 + dy)))
```

```

def plate_coverage(preds, conf, pad=BLUR_PAD):
    """Per labelled plate: its pixel box, width and the share of its area covered by the plate
    rows, n_boxes = [], 0
    for stem in test_stems:
        row = images_df[(images_df['split'] == 'test') & (images_df['stem'] == stem)]
        boxes, confs = preds[stem]
        kept = [pad_box(b, pad, row['img_w'], row['img_h']) for b, c in zip(boxes, confs)]
        n_boxes += len(kept)
        for gx1, gy1, gx2, gy2 in gt_boxes(stem):
            mask = np.zeros((max(gy2 - gy1, 1), max(gx2 - gx1, 1)), bool)
            for bx1, by1, bx2, by2 in kept:
                ix1, iy1, ix2, iy2 = max(gx1, bx1), max(gy1, by1), min(gx2, bx2), min(gy2, by2)
                if ix2 > ix1 and iy2 > iy1:
                    mask[iy1 - gy1:iy2 - gy1, ix1 - gx1:ix2 - gx1] = True
            rows.append({'stem': stem, 'box': (gx1, gy1, gx2, gy2), 'width': row['img_w'],
                        'share': mask.mean()})
    return pd.DataFrame(rows), n_boxes

def coverage_report(preds, conf, pad=BLUR_PAD):
    """Counts of fully covered, partly covered and missed plates at `conf`."
    per_plate, n_boxes = plate_coverage(preds, conf, pad)
    shares = per_plate['share'].to_numpy()
    return {'threshold': conf, 'blur boxes': n_boxes, 'plates': len(shares),
            'fully covered': int((shares >= 0.9).sum()),
            'partly covered': int(((shares >= 0.1) & (shares < 0.9)).sum()),
            'missed': int((shares < 0.1).sum()),
            'fully covered %': round((shares >= 0.9).mean() * 100, 1)}

client_preds = predict_test(model)
pilot_preds = predict_test(pilot_model)

coverage = pd.DataFrame(
    [{'model': 'Pilot model', **coverage_report(pilot_preds, BLUR_CONF)}]
    + [{'model': 'Client model', **coverage_report(client_preds, t)} for t in thresholds])
coverage.set_index(['model', 'threshold'])

```

Out [22]:

		blur boxes	plates	fully covered	partly covered	missed	fully covered %
model	threshold						
Pilot model	0.2	626	512	393	22	97	76.8
Client model	0.1	594	512	467	18	27	91.2
	0.2	531	512	462	16	34	90.2
	0.3	502	512	450	10	52	87.9
	0.5	445	512	424	5	83	82.8

In [23]: # Visual 1: plates fully covered, partly covered and missed, per model and threshold

```

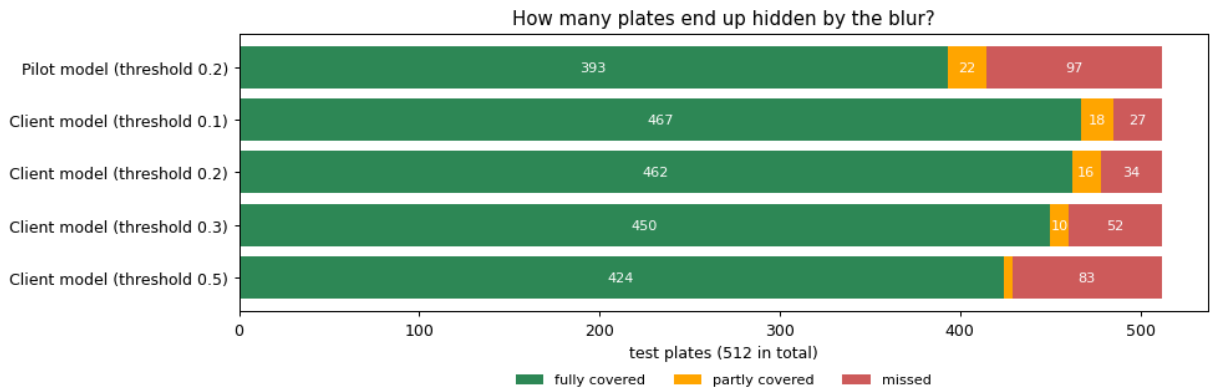
labels = [f'{m} (threshold {t})' for m, t in coverage.index]

```

```

fig, ax = plt.subplots(figsize=(11, 3.8))
left = np.zeros(len(coverage))
for col, color in [('fully covered', 'seagreen'), ('partly covered', 'orange'), ('missed', 'firebrick')]:
    ax.barh(labels, coverage[col], left=left, color=color, label=col)
    for i, (l, v) in enumerate(zip(left, coverage[col])):
        if v >= 8:
            ax.text(l + v / 2, i, str(v), ha='center', va='center', color='white')
    left += coverage[col].to_numpy()
ax.invert_yaxis()
ax.set_xlabel('test plates (512 in total)')
ax.set_title('How many plates end up hidden by the blur?')
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.18), ncol=3, fontsize=10)
plt.tight_layout()
plt.show()

```

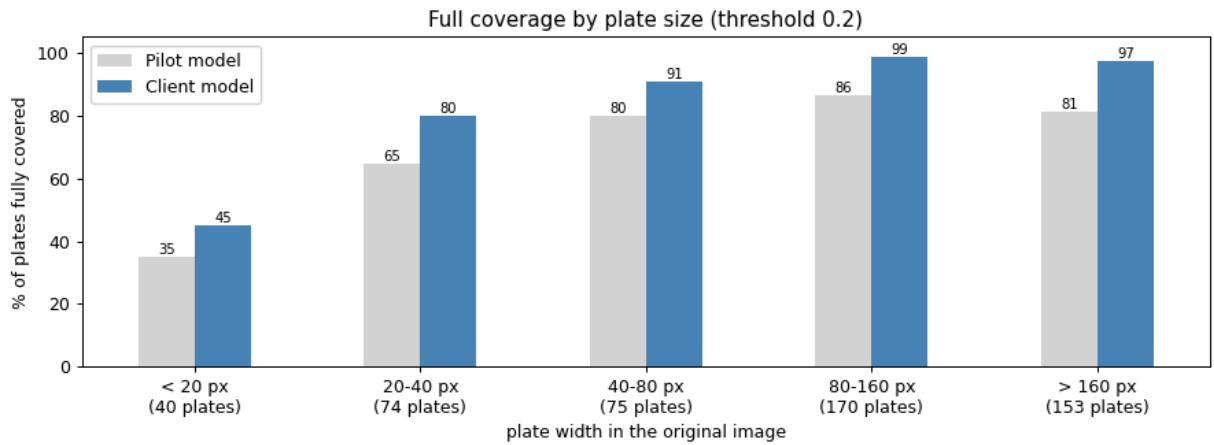


```

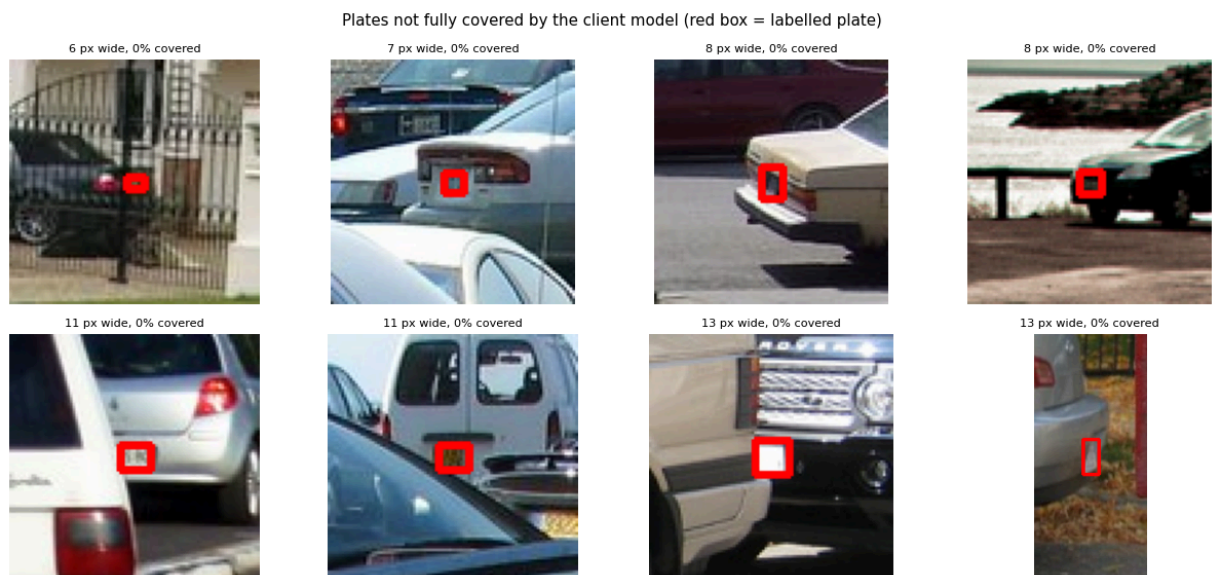
In [24]: # Visual 2: full coverage by plate size, pilot vs client model at the deploy
size_bins = [0, 20, 40, 80, 160, 10000]
size_labels = ['< 20 px', '20-40 px', '40-80 px', '80-160 px', '> 160 px']
by_size = {}
for name, preds in [('Pilot model', pilot_preds), ('Client model', client_preds)]:
    per_plate, _ = plate_coverage(preds, BLUR_CONF)
    per_plate['size'] = pd.cut(per_plate['width_px'], size_bins, labels=size_labels)
    by_size[name] = per_plate.groupby('size', observed=False)['share'].apply(lambda x: x.sum())
    if name == 'Client model':
        client_per_plate = per_plate
by_size = pd.DataFrame(by_size)
counts = client_per_plate['size'].value_counts().reindex(size_labels)

ax = by_size.plot.bar(figsize=(10, 3.8), rot=0, color=['lightgray', 'steelblue'])
ax.set_xticklabels([f'{l}\n({counts[l]} plates)' for l in size_labels])
ax.set_ylim(0, 105)
ax.set_ylabel('% of plates fully covered')
ax.set_xlabel('plate width in the original image')
ax.set_title(f'Full coverage by plate size (threshold {BLUR_CONF})')
for container in ax.containers:
    ax.bar_label(container, fmt='%.0f', fontsize=8)
plt.tight_layout()
plt.show()

```



```
In [25]: # Visual 3: the plates the client model still misses, shown with surrounding
missed = client_per_plate[client_per_plate['share'] < 0.9].sort_values('width')
if len(missed):
    cols = 4
    rows_n = int(np.ceil(len(missed) / cols))
    fig, axes = plt.subplots(rows_n, cols, figsize=(14, 3.2 * rows_n), squeeze=True)
    for ax, (_, m) in zip(axes.flat, missed.iterrows()):
        image = load_rgb(os.path.join(image_dir('test'), m['stem'] + '.jpg'))
        x1, y1, x2, y2 = m['box']
        mx, my = max(40, (x2 - x1) * 2), max(40, (y2 - y1) * 3)
        cx1, cy1 = max(0, x1 - mx), max(0, y1 - my)
        crop = image[cy1:min(image.shape[0], y2 + my), cx1:min(image.shape[1], x2 + mx)]
        cv2.rectangle(crop, (x1 - cx1, y1 - cy1), (x2 - cx1, y2 - cy1), (255, 0, 0))
        ax.imshow(crop)
        ax.set_title(f"{m['width_px']} px wide, {m['share'] * 100:.0f}% covered")
    for ax in axes.flat:
        ax.axis('off')
    fig.suptitle('Plates not fully covered by the client model (red box = labelled plate)')
    plt.tight_layout()
    plt.show()
```



Interpretation

- At the configured threshold of 0.2, the client model fully covers **462 of 512 plates (90.2%)**, partly covers 16 and misses 34. The pilot model, without client data, misses 97 and leaves 22 partly covered.
- **Lowering the threshold helps privacy.** At 0.1 full coverage is 91.2% (27 missed) at the cost of 63 extra blur boxes, most of them on harmless regions. A "high-confidence only" policy at 0.5 would leave **83 plates visible**, 49 more than at 0.2. For a privacy control this trade-off is clearly in favour of a low threshold; the threshold should be re-tuned whenever the model is retrained.
- **Remaining misses are mainly small, distant plates.** The size chart shows that coverage falls sharply for plates under about 40 px wide, as the plate-size analysis in Section 4.3 anticipated. Since validation scores levelled off during training (Section 8), more real-world training data with small, distant plates is the lever for these cases.

10. Anonymisation: Blurring Only Inside the Detected Box

How the blur is confined to the plate. For every detection the pipeline:

1. Takes the box corners `(x1, y1, x2, y2)` predicted by YOLO (already in pixels).
2. Enlarges the box by 15% on each side and clips it to the image, so a slightly tight prediction cannot leave the edge of the plate readable.
3. Selects only that region with array slicing, `image[y1:y2, x1:x2]`, applies a Gaussian blur to this region alone, and writes the result back into the same position.
4. Scales the blur kernel with the plate size (about the full plate width), so a large close-up plate is blurred as strongly as a small distant one.

Pixels outside the boxes are never modified, so the rest of the image keeps its full value for analytics and model training. The blur is applied to a copy, which keeps the original available for any authorised processing.

```
In [26]: def blur_plates(image_rgb, boxes, pad=BLUR_PAD):
    """Returns a copy of the image with a Gaussian blur inside each (padded)
    out = image_rgb.copy()
    img_h, img_w = out.shape[:2]
    for box in boxes:
        x1, y1, x2, y2 = pad_box(box, pad, img_w, img_h)
        roi = out[y1:y2, x1:x2] # a view: writing to it changes `out` in place
        if roi.size == 0:
            continue
        k = max(roi.shape[:2]) // 2 * 2 + 1 # odd kernel about the size of
        roi[:] = cv2.GaussianBlur(roi, (k, k), 0)
    return out

def anonymise(image_path, m=model, conf=BLUR_CONF):
    """Detects plates and returns (original, detected boxes with confidence,
```

```

image = load_rgb(image_path)
r = m.predict(image_path, conf=conf, iou=0.5, imsz=IMGSZ, device=DEVICE)
boxes = r.bboxes.xyxy.cpu().numpy().astype(int)
return image, list(zip(boxes, r.bboxes.conf.cpu().numpy())), blur_plates(

demo = pick('test', min_plates=1, n=2) + pick('test', min_plates=3, n=2)
fig, axes = plt.subplots(len(demo), 2, figsize=(13, 4.2 * len(demo)))
for (ax_det, ax_blur), stem in zip(axes, demo):
    original, detections, blurred = anonymise(os.path.join(image_dir('test'))
    annotated = original.copy()
    thickness = max(2, original.shape[1] // 400)
    for (x1, y1, x2, y2), c in detections:
        cv2.rectangle(annotated, (x1, y1), (x2, y2), (0, 255, 0), thickness)
        cv2.putText(annotated, f'{c:.2f}', (x1, max(y1 - 5, 15)), cv2.FONT_F
            thickness * 0.4, (255, 0, 0), thickness)
    ax_det.imshow(annotated)
    ax_det.set_title(f'Detections ({len(detections)})', fontsize=10)
    ax_blur.imshow(blurred)
    ax_blur.set_title('Anonymised output', fontsize=10)
    ax_det.axis('off')
    ax_blur.axis('off')
plt.tight_layout()
plt.show()

```

Detections (1)



Anonymised output



Detections (2)



Anonymised output



Detections (3)



Anonymised output



Detections (5)



Anonymised output



10.1 Blur effectiveness: can OCR still read the plates?

We run the Tesseract OCR engine on each sufficiently large labelled test plate, once on the original image and once on the anonymised image, and count how often it still

returns a plate-like string (three or more letters or digits).

```
In [27]: try:
import pytesseract
pytesseract.pytesseract.tesseract_cmd = shutil.which('tesseract') or '/c
pytesseract.get_tesseract_version()
OCR_AVAILABLE = True
except Exception as e:
OCR_AVAILABLE = False
print('Tesseract not available, skipping the OCR check:', e)

def ocr_chars(crop):
    """Number of letters and digits Tesseract reads in a plate crop."""
    text = pytesseract.image_to_string(crop, config='--psm 7')
    return sum(ch.isalnum() for ch in text)

if OCR_AVAILABLE:
    rows = []
    for stem in test_stems:
        plates = [b for b in gt_boxes(stem) if b[2] - b[0] >= 60] # large e
        if not plates:
            continue
        original, _, blurred = anonymise(os.path.join(image_dir('test'), ste
        for x1, y1, x2, y2 in plates:
            rows.append({'original': ocr_chars(original[y1:y2, x1:x2]) >= 3,
                        'anonymised': ocr_chars(blurred[y1:y2, x1:x2]) >= 3
        ocr = pd.DataFrame(rows)
        display(pd.DataFrame({'plates checked': [len(ocr)] * 2,
                             'OCR returns 3+ characters': [ocr['original'].sum(
                                index=['original image', 'anonymised image'])])
```

	plates checked	OCR returns 3+ characters
original image	350	141
anonymised image	350	13

Interpretation. On the original images Tesseract reads a plate-like string from 141 of 350 legible plates (40%). After anonymisation this drops to 13 (4%), a 91% reduction. The remaining readings most likely come from plates the detector missed or only partly covered (Section 9.1) rather than from weak blurring, since the blur kernel spans the full plate width. Tesseract is a document OCR engine and reads fewer than half of the plates even before blurring, so this check is a sanity test; a stricter audit would use a plate-specific OCR model.

11. Answers to the Business Questions

Q1. How do you convert normalised YOLO coordinates into pixel coordinates for OpenCV? Multiply the normalised centre and size by the image width and height, then

move from the centre to the corners by half the width and height: $x1 = (xc - w/2) \cdot W$, $y1 = (yc - h/2) \cdot H$, $x2 = (xc + w/2) \cdot W$, $y2 = (yc + h/2) \cdot H$, rounded to integers and clipped to the image. Section 5 implements this as `yolo_to_pixel` and verifies it on every validation and test plate.

Q2. Write a function to overlay bounding boxes onto raw images. Why is this step critical before training? `draw_yolo_boxes` in Section 6 overlays the labels. The step is critical because the model learns exactly what the labels say: format errors and missing annotations do not raise errors, they silently produce a model that misses plates. In the pilot study this check revealed that 8% of plates had been dropped during label conversion.

Q3. How did annotation quality impact the final blurring accuracy? Blurring can only be as good as detection, and detection is only as good as the labels:

- **Completeness.** A plate without a label is learned as background, which directly produces missed, unblurred plates. In the pilot study, a label conversion that kept one plate per image dropped 8% of plates; this had to be fixed before any metric could be trusted.
- **Duplicates and leakage.** 155 duplicate files and 113 training copies of validation photos would have over-weighted repeated images and inflated validation scores, hiding real weaknesses.
- **Representativeness.** Only about 19% of training images resemble the real-world test photos, and 83% are augmented copies. This domain gap is the main reason why small, distant plates are still missed (34 of 512 at the configured threshold).
- **Box tightness.** mAP@50-95 (0.682) is lower than mAP@50 (0.921), so predicted boxes are sometimes slightly off the plate edges. The 15% padding around each box compensates, so a slightly misaligned box still covers the whole plate.

Q4. How did you preprocess images without losing fine-grained plate details?

Aspect-ratio-preserving letterbox resizing to 640 px (chosen from the plate-size analysis), no lossy transformations, mosaic augmentation to teach small-plate detection, and removal of duplicates and leaking images. Details in Section 7.4.

Q5. Why YOLOv8 instead of a two-stage detector? A single forward pass gives millisecond-level inference, simple deployment to edge devices and strong accuracy on a single, visually distinctive class. A two-stage detector would cost several times more compute for little practical gain on this task. Details in Section 7.1.

Q6. Which YOLOv8 variant did you select, and how does it balance speed and accuracy? YOLOv8s. It has enough capacity for small, distant and multiple plates in full street scenes while running in a few milliseconds per image (measured in Section 9). Larger variants would multiply training and inference cost for gains smaller than those available from better data. Details in Section 7.2.

Q7. How did license plate characteristics influence the choice of model scale?

Plates are small (one in ten test plates is under about 23 px wide), wide and short, often several per image, but rigid and high-contrast. Their small size drove the **640 px input resolution**; their simple, consistent appearance meant a **small model** is sufficient. Details in Section 7.3.

Q8. How did you apply blurring only within the detected bounding box? Each predicted box is enlarged by 15%, clipped to the image, and only that slice of the image array is replaced by a Gaussian-blurred version of itself, with a kernel scaled to the plate size. All other pixels are untouched. Details in Section 10.

12. Insights and Recommendations

Key insights

1. **Recall is the business metric.** A missed plate is a compliance failure, an extra blur is not. Thresholds, padding and evaluation were therefore designed around plates left visible, not around mAP alone.
2. **Client data pays off.** Training on client data raises mAP@50-95 from 0.442 to 0.682 (+54%), full plate coverage from 76.8% to 90.2%, and cuts plates left visible from 97 to 34, while inference time falls from 3.9 ms to 2.9 ms per image.
3. **Small plates drive the design.** One in ten test plates is under about 23 px wide and most remaining misses are under 40 px, so 640 px input is essential; at 320 px a large share of plates would be invisible to the model.
4. **The data, not the model or training time, is the next lever.** Validation scores levelled off after about 40 of 90 epochs. The training mix is dominated by close-up and augmented images, while real use involves full street scenes with several, often small, plates per image.

Recommendations

1. **Retrain when new data arrives, not for more epochs.** About 40 epochs are sufficient on this data (roughly 10 hours on a laptop GPU). The notebook reproduces every figure from new weights by changing one setting (`TRAIN_RUN_NAME`).
2. **Deploy with a low threshold (0.1 to 0.2) and 15% padding**, and log every detection between the threshold and 0.5 for **human review**; confirmed misses go back into the training set (continuous learning, as proposed in the client's approach document).
3. **Add real-world street photos** with complete, multi-plate labels, especially small, distant, night and rain scenes, and keep validation and test free of near-duplicates.
4. **Use plate coverage as a release gate:** a new model ships only if the share of fully covered plates on the fixed test set does not fall.
5. **Edge deployment.** Export the model to ONNX/TensorRT (or CoreML on Apple hardware) to anonymise on the device, so raw plates never leave the camera.

